

XIV. Introduction à Bootstrap

BOOTSTRAP EN UNE PHRASE

Un framework CSS qui fournit une **GRILLE DE MISE EN PAGE** et une **bibliothèque de composants prêts à l'emploi** (boutons, navbars, cards, formulaires) pour construire rapidement des interfaces cohérentes et responsives.

Sommaire

- **Introduction**
 - 1. Objectifs du chapitre
 - 2. En pratique — situation professionnelle
 - 3. Quand l'utiliser, quand l'éviter
- **A. Qu'est-ce qu'un framework CSS ?**
- **B. Installation de Bootstrap**
 - 1. CDN (recommandé pour commencer)
 - 2. Via npm (projets professionnels)
 - 3. Vérifier que Bootstrap est actif
- **C. Les 3 piliers de Bootstrap**
 - 1. La grille 12 colonnes (chapitre XV)
 - 2. Les utilities (chapitre XV aussi)
 - 3. Les composants pré-stylés (chapitre XVI)
- **D. Quand utiliser Bootstrap ? Quand ne pas ?**
 - Bootstrap excelle pour
 - Bootstrap à nuancer pour
 - Alternative : Tailwind CSS
- **Conclusion**
 - 1. À retenir
 - 2. Pour aller plus loin

Introduction

1. Objectifs du chapitre

À l'issue de ce chapitre, vous serez capable de :

- **Expliquer** ce qu'est Bootstrap et dans quels cas il fait gagner du temps.
- **Installer** Bootstrap dans un projet via CDN (solution la plus simple) ou npm.
- **Reconnaître** la philosophie **utility-first** + **composants pré-stylés** de Bootstrap.
- **Distinguer** les cas où Bootstrap est pertinent (rapidité, cohérence, admin/dashboard) des cas où il ne l'est pas (site vitrine custom, performance critique).

2. En pratique — situation professionnelle

Votre agence doit livrer un dashboard d'administration pour un client en **2 semaines**. Tout from-scratch prendrait 4 semaines. Bootstrap vous apporte **instantanément** : système de grille 12 colonnes, composants navbar/sidebar/tables/modals, thème cohérent — gain de 60 % de temps pour un résultat pro.

3. Quand l'utiliser, quand l'éviter

✅ Utiliser pour	❌ Éviter quand
Dashboard / back-office / MVP livré en quelques jours	Site vitrine avec design signature (on écrase Bootstrap)
Équipe mixte dev / non-dev (doc Bootstrap = langage commun)	Perf critique (~150 kB CSS, réductible mais lourd)
Prototypes et démos client (livrer en 1 jour)	Apprendre les bases CSS (Bootstrap masque le natif)

A. Qu'est-ce qu'un framework CSS ?

Un **framework CSS** est une bibliothèque pré-écrite qui fournit :

1. **Un reset / base** (comme le `*, *::before, *::after { box-sizing: border-box }` qu'on apprend à écrire).
2. **Un système de grille** responsive (colonnes, breakpoints).
3. **Des classes utilitaires** (`.p-4` , `.text-center` , `.d-flex`).
4. **Des composants pré-stylés** (boutons, cards, navbars, modals).
5. **Un thème cohérent** (couleurs, typographie, espacements par défaut).

Principaux frameworks en 2026 :

Framework	Approche	Popularité
Bootstrap	Composants + utilities	Ultra-dominant sur les admin/dashboards
Tailwind CSS	Utility-first strict (atomique)	En forte croissance, sites customs
Bulma	Composants purs CSS (sans JS)	Alternative légère
Foundation	Composants + grille flexible	Moins utilisé aujourd'hui

Bootstrap reste le framework **le plus utilisé** : Open Source, Twitter en 2011, maintenu activement, support navigateur très large.

B. Installation de Bootstrap

1. CDN (recommandé pour commencer)

Dans le `<head>` de votre HTML :

```
←!— CSS →  
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"  
      rel="stylesheet">
```

Juste avant `</body>` :

```
←!— JS (pour modals, dropdowns, toasts, carousel...) →  
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></scr  
ipt>
```

✅ **Avantages CDN** : aucune installation, cache partagé avec d'autres sites, commence à coder en 10 secondes. ❌ **Inconvénients** : dépendance externe, taille non optimisable, customisation limitée.

2. Via npm (projets professionnels)

```
npm install bootstrap
```

Puis dans votre `main.scss` (avec un bundler type Vite) :

```
@import "bootstrap/scss/bootstrap";
```

Ou importer seulement ce dont on a besoin pour alléger le bundle :

```
@import "bootstrap/scss/functions";  
@import "bootstrap/scss/variables";  
@import "bootstrap/scss/mixins";  
@import "bootstrap/scss/grid";  
@import "bootstrap/scss/buttons";  
// ...sélection des modules nécessaires
```

✅ **Avantages npm** : personnalisation via variables SCSS, tree-shaking, bundle optimisé.

3. Vérifier que Bootstrap est actif

Ajoutez ce bouton dans votre HTML :

```
<button class="btn btn-primary">Test</button>
```

Si le bouton apparaît en **bleu arrondi** (au lieu du bouton natif gris), Bootstrap fonctionne .

C. Les 3 piliers de Bootstrap

1. La grille 12 colonnes (chapitre XV)

Système de mise en page **responsive** basé sur 12 colonnes.

```
<div class="container">
  <div class="row">
    <div class="col-md-6">Gauche (6/12 colonnes sur tablette+)</div>
    <div class="col-md-6">Droite</div>
  </div>
</div>
```

2. Les utilities (chapitre XV aussi)

Classes atomiques qui font **une seule chose**. Exemples :

```
<div class="p-4 mt-5 bg-primary text-white text-center rounded-3 shadow">
  Je suis stylé sans une ligne de CSS
</div>
```

Classe	Effet
<code>.p-4</code>	<code>padding: 1.5rem;</code>
<code>.mt-5</code>	<code>margin-top: 3rem;</code>
<code>.bg-primary</code>	<code>background: #0d6efd;</code>
<code>.text-white</code>	<code>color: white;</code>
<code>.text-center</code>	<code>text-align: center;</code>
<code>.rounded-3</code>	<code>border-radius: 0.5rem;</code>
<code>.shadow</code>	<code>box-shadow</code> standard

→ Philosophie **utility-first** : vous construisez l'UI en combinant des classes, sans écrire de CSS custom.

3. Les composants pré-stylés (chapitre XVI)

```
<nav class="navbar navbar-expand-lg bg-light">...</nav>  
<div class="card">...</div>  
<form class="needs-validation">...</form>  
<div class="modal fade">...</div>
```

→ Chaque composant a son **balisage structuré** et ses **classes cohérentes** — reproduire à l'identique pour avoir le rendu attendu.

D. Quand utiliser Bootstrap ? Quand ne pas ?

Bootstrap excelle pour

- **Dashboards, back-offices, applications internes** — vitesse + cohérence.
- **Prototypes et MVP** — livrer en 1 jour ce qui prendrait 1 semaine.
- **Équipes mixtes dev/non-dev** — la doc Bootstrap est un langage commun.
- **Clients qui veulent *ça marche, vite*** sans design custom poussé.

Bootstrap à nuancer pour

- **Sites vitrines avec design très custom** — on finit par écraser Bootstrap, donc autant pas l'utiliser.
- **Optimisation performance extrême** — même minifié, Bootstrap fait ~150 kB de CSS (réductible avec npm + tree-shaking).
- **Projets qui veulent un *look signature*** — beaucoup de sites Bootstrap se ressemblent si on ne customise pas.

Alternative : Tailwind CSS

Tailwind CSS est **utility-first strict** (pas de composants pré-stylés). Plus de liberté créative, mais demande d'écrire plus de classes. Voir chapitre XVIII pour comparaison Bootstrap vs Tailwind vs CSS custom.

♦ Notre cours mise sur Bootstrap parce que...

1. C'est le framework le plus répandu dans les entreprises françaises en 2026.
2. Il est **pédagogique** : les classes utilitaires reflètent directement les propriétés CSS qu'on vient d'apprendre (`.p-4` = padding, `.d-flex` = display: flex, `.text-center` = text-align: center).
3. Il prépare naturellement à d'autres frameworks (Tailwind suivra la même logique utility-first).

Conclusion

1. À retenir

- Bootstrap = **grille 12 colonnes** + **utilities** + **composants pré-stylés** + thème cohérent.
- Installation la plus simple : **CDN** dans le `<head>`.
- Utilisation pro : **npm + SCSS** pour personnaliser et optimiser.
- Philosophie : **utility-first** (classes atomiques) + composants pré-stylés.
- Idéal pour : dashboards, prototypes, MVP. À nuancer pour : sites design custom, perf critique.
- En 2026, **Bootstrap reste le framework CSS le plus utilisé** en entreprise française.

2. Pour aller plus loin

- **Bootstrap — site officiel**
- **Bootstrap Docs — Get started**
- **Tailwind CSS (alternative)**
- **Bulma (alternative sans JS)**